POCKET SMART AI

Smart Budget Planning & Personalized Recommendations

Project Report

Team Information:

• Afzal Z** (Team Lead)

• Avinash N

• Vishva R

• Sangeetha S

• Raghul Kumar S

Prepared for Academic Project Submission

Table of Contents

1. [Project Title](#1-project-title)
2. [Project Description](#2-project-description)
3. [Application Scenarios / Use Cases](#3-application-scenarios--use-cases)
4. [Technical Architecture Overview](#4-technical-architecture-overview)
5. [Prerequisites](#5-prerequisites)
6. [Prior Knowledge Required](#6-prior-knowledge-required)
7. [Project Objectives](#7-project-objectives)
8. [System Workflow](#8-system-workflow)
9. [Milestone 1 - Requirement Analysis & System Design](#9-milestone-1---requirement-analysis--system-design)
10. [Milestone 2 - Environment Setup & Initial Configuration](#10-milestone-2---environment-setup--initial-configuration)
11. [Milestone 3 - Core System Development](#11-milestone-3---core-system-development)
12. [Milestone 4 - Integration & Optimization](#12-milestone-4---integration--optimization)
13. [Milestone 5 - Testing & Validation](#13-milestone-5---testing--validation)
14. [Deployment](#14-deployment)
15. [Project Structure](#15-project-structure)
16. [Results](#16-results)
17. [Advantages & Limitations](#17-advantages--limitations)
18. [Future Enhancements](#18-future-enhancements)
19. [Conclusion](#19-conclusion)
20. [Appendix](#20-appendix)

---

1. Project Title

Project Title: POCKET SMART AI - Smart Budget Planning & Personalized Recommendations

Project Type: Web-based budgeting and recommendation application.

Project Purpose: To help a user turn a fixed budget into a practical plan by collecting preferences, processing requirements, and presenting organized recommendations. The project is designed around three planning areas: Home, Party, and Jewelry.

2. Project Description

Pocket Smart AI is a browser-based application developed to make budget-based planning easier. Instead of requiring a user to manually compare many choices, the application asks for the information that matters to the selected planning category and produces a structured recommendation result.

The project was implemented as a FastAPI application with a lightweight web interface. Users can create an account, log in, reach a dashboard, select a planner, enter their requirements, and review generated recommendations. The system also keeps recommendation history for the logged-in user.

Main Capabilities:

* Account registration and login with session-based access.
* Dashboard showing the logged-in user and saved recommendation count.
* Home Planner for room/home requirements, budget, style, and item quantities.
* Party Planner for event budget, guest count, event type, venue preference, location, and additional preferences.
* Jewelry Planner for occasion, budget, style, and outfit-related input.
* Recommendation results with platform/source, item title, reason, estimated price, and a link.
* History page for previously generated plans.
* Gemini AI integration when configured, with a fallback recommendation engine when AI access is unavailable.
* Responsive browser interface using HTML templates, CSS, and JavaScript.

The application is intended to support planning decisions before spending money. It does not act as a payment system and the displayed product or service information is treated as recommendation data rather than a live purchase transaction.

3. Application Scenarios / Use Cases

* **Use Case 1 - Home Planning:** A user enters a total amount available for a room or home requirement, selects a preferred style, and identifies the items needed. Pocket Smart AI processes these details and produces a budget-conscious set of suggestions.
* **Use Case 2 - Party Planning:** A user preparing a birthday or another event can provide the guest count, event type, venue preference, location, and personal preferences. The system converts these inputs into a structured event recommendation.
* **Use Case 3 - Jewelry Planning:** A user preparing for an occasion such as a wedding can specify a budget, occasion, desired style, and outfit-related information. The planner then presents suitable recommendation choices within the requested spending range.
* **Use Case 4 - Recommendation History:** After a plan is generated, the user can return to the History section and review previously saved planning results. This makes the application more useful than a one-time recommendation form.
* **Use Case 5 - Academic Demonstration:** The project can demonstrate practical concepts including authentication, database operations, API routes, form validation, server-side rendering, client-side requests, recommendation logic, and AI-assisted application design.

4. Technical Architecture Overview

The implemented system follows a layered web-application structure. A browser provides the interface, FastAPI handles HTTP requests and application routing, service modules contain recommendation-related logic, and SQLite provides persistent local storage.

* **Frontend / Presentation:** Jinja2 HTML templates, CSS styling, and JavaScript for browser-side requests and dynamic result rendering.
* **Backend:** FastAPI routes for authentication, pages, planners, and supporting API operations.
* **Authentication:** Password hashing and signed access tokens stored in an HTTP-only cookie.
* **Database:** SQLite for user records and saved recommendation plans.
* **Recommendation Layer:** Catalog/fallback logic plus optional Gemini AI generation.
* **Configuration:** Environment variables are used for secrets, database settings, allowed origins, and AI configuration.
* **Static Resources:** CSS and JavaScript are served from the application's static directory.

The project uses a modular structure so that page routing, authentication, database access, recommendation services, and planner endpoints are separated instead of being placed into one large application file.

5. Prerequisites

Software Requirements:

* Windows laptop or desktop for development.
* Python 3.12.x environment.
* Visual Studio Code or another Python-capable editor.
* Git for source control and GitHub for repository hosting.
* Modern web browser such as Chrome or Edge.
* Internet connection when installing packages or using the optional Gemini service.

Project Dependencies:

The project dependency file includes FastAPI, Uvicorn, Jinja2, python-multipart, python-dotenv, Pydantic, pydantic-settings, PyJWT, Passlib/Bcrypt, Google GenAI, pytest, and HTTPX. The repository's `requirements.txt` is the authoritative dependency list for the submitted build.

6. Prior Knowledge Required

* **Python fundamentals:** variables, functions, modules, exceptions, and basic object-oriented concepts.
* **Web fundamentals:** HTML, CSS, JavaScript, forms, HTTP requests, and browser navigation.
* **FastAPI concepts:** routes, request handling, dependency injection, templates, and JSON responses.
* **Database basics:** tables, records, SQL queries, and persistent storage.
* **Authentication concepts:** registration, password hashing, cookies, sessions/tokens, and protected routes.
* **Git/GitHub basics:** repositories, commits, branches, and source-code sharing.
* **Basic AI concepts:** prompts, model responses, fallback behavior, and handling optional AI services.

7. Project Objectives

* Build a usable web application that combines budgeting with personalized recommendations.

* Give users separate planning experiences instead of forcing all requirements into one generic form.
* Keep user accounts and generated plans persistent so recommendations can be reviewed later.
* Provide clear validation and error handling for incomplete or incorrect input.
* Organize the code into maintainable modules for pages, APIs, authentication, database access, and recommendation services.
* Support AI-generated recommendations while retaining a fallback path when the AI service is not configured or unavailable.
* Create a project that can be demonstrated locally and maintained through a GitHub repository.

8. System Workflow

1. The user opens Pocket Smart AI and chooses registration or login.
2. After successful authentication, the application creates an access session using a secure HTTP-only cookie.
3. The dashboard loads the user's session information and saved recommendation count.
4. The user selects Home, Party, or Jewelry Planner.
5. The chosen form collects the budget and category-specific requirements.
6. The backend validates the request and sends the information to the recommendation layer.
7. The system produces recommendations using the configured AI service or fallback logic.
8. The result is displayed with estimated prices, reasons, source/platform information, and links.
9. The completed recommendation is stored in the database for the user's history.
10. The user can revisit the History page to inspect earlier plans or log out.

9. Milestone 1 - Requirement Analysis & System Design

9.1 Problem Definition

People often know how much they can spend but are unsure how to divide that amount across several related needs. The project addresses this gap by combining budget input, preferences, category-specific questions, and recommendation output in one workflow.

9.2 Functional Requirements

* **FR-01:** Provide landing, registration, login, and dashboard pages.
* **FR-02:** Allow a user to create an account and authenticate.
* **FR-03:** Accept a budget and planner-specific requirements.
* **FR-04:** Validate user input before recommendation processing.
* **FR-05:** Generate structured recommendations.
* **FR-06:** Display estimated spending and recommendation reasons.
* **FR-07:** Save generated plans for authenticated users.
* **FR-08:** Provide a history view.
* **FR-09:** Handle invalid requests and unavailable AI services without crashing the application.

9.3 Non-Functional Requirements

* Usability
* Maintainable modular code
* Basic authentication and secret protection
* Predictable validation
* Responsive browser interaction

* Reasonable local performance

* Reproducible setup from the repository

9.4 Design Decisions

The system was divided into routers, services, database/authentication modules, templates, and static assets. This separation made debugging easier during development and allowed problems such as route conflicts, template recursion, and session handling to be isolated without rewriting the entire application.

10. Milestone 2 - Environment Setup & Initial Configuration

Development was carried out in Visual Studio Code on Windows using Python 3.12.2 and a project-specific virtual environment. Dependencies were installed from `requirements.txt` and the FastAPI application was started with Uvicorn during local testing.

* Created and activated a Python virtual environment.

* Installed the project's dependency set.

* Configured environment variables through `.env` while keeping secrets outside GitHub.

* Initialized the SQLite database used by the application.

* Started the FastAPI server locally and verified browser access.

* Initialized a Git repository and committed the working project.

* Created the GitHub repository and uploaded the non-secret project files.

The project intentionally excludes the local `.env` file, virtual environment, and SQLite database from the public repository because these are environment-specific or generated runtime data.

11. Milestone 3 - Core System Development

11.1 Authentication and User Access

Registration stores a user account with a protected password representation. Login validates the supplied credentials and creates an HTTP-only access cookie. Protected API operations use that session information to identify the current user.

11.2 Dashboard

The dashboard presents the logged-in email and the number of recommendation plans saved. It also provides entry points to the three planners and the history page.

11.3 Home Planner

The Home Planner accepts a budget, room context, design preference, and item requirements. The tested workflow successfully produced a recommendation set for a 10,000 example containing a bed, lamp, and table requirements.

11.4 Party Planner

The Party Planner collects budget, guest count, event type, venue preference, location, and additional preferences. A tested birthday planning flow with a 10,000 budget generated recommendations successfully.

11.5 Jewelry Planner

The Jewelry Planner collects the occasion, budget, style, and outfit-related input. A wedding/elegant example with a ₹10,000 budget successfully returned recommendations.

11.6 History

Generated plans are stored against the authenticated user. The History page retrieves saved records through the application's history API and displays the planner type, plan number, and creation time.

12. Milestone 4 - Integration & Optimization

Integration connected the HTML pages with the FastAPI endpoints and database operations. Browser-side JavaScript sends requests with credentials enabled, processes JSON responses, and renders recommendation cards dynamically.

Major Integration Fixes Completed:

* Resolved a login-page JavaScript reference issue so the login form correctly redirects to the dashboard.
* Removed a recursive template problem in `base.html` that prevented pages from rendering correctly.
* Corrected dashboard JavaScript so session information and recommendation counts load properly.
* Separated the JSON history endpoint from the HTML history page to avoid a route collision.
* Updated dashboard and navigation links to the corrected history page route.
* Added escaping for displayed recommendation values to reduce unsafe HTML injection through rendered data.
* Kept Gemini optional by allowing the fallback recommendation engine to operate without an API key.

Optimization Approach:

The application keeps planner logic separated by category, avoids unnecessary page reloads for API-driven results, and uses a fallback path for recommendation generation. Configuration values and secrets are kept outside the committed source code.

13. Milestone 5 - Testing & Validation

13.1 Functional Testing Performed

ID	Test	Expected / Observed Result	Status
TC-01	Open application	Landing page loaded	Passed
TC-02	Register new user	Account created	Passed
TC-03	Login with valid credentials	Dashboard opened	Passed
TC-04	Dashboard session data	Email and recommendation count displayed	Passed
TC-05	Home Planner request	Recommendations generated and saved	Passed
TC-06	Party Planner request	Recommendations generated and saved	Passed
TC-07	Jewelry Planner request	Recommendations generated and saved	Passed
TC-08	History page	Saved plans displayed with timestamps	Passed
TC-09	Logout and re-login	Session ended and could be recreated	Passed
TC-10	GitHub source submission	Project source uploaded without local secrets	Passed

The functional checks above reflect the development/testing work completed on the submitted local build. Formal load testing and production security testing were not performed and therefore are not reported as measured results.

14. Deployment

At the current project stage, Pocket Smart AI has been verified as a local FastAPI web application and its source has been placed in a GitHub repository. A public production deployment URL has not been claimed in this report.

Local Deployment:

1. Create/activate the Python virtual environment.
2. Install the dependencies from `requirements.txt`.
3. Configure the required `.env` values.
4. Start the FastAPI application with Uvicorn.
5. Open the local application URL in a browser and perform the smoke tests.

Future Hosted Deployment:

For a public demonstration, the project can be deployed to a suitable Python hosting platform. The production environment would need the environment variables, database strategy, HTTPS, application start command, and any required Gemini credentials configured securely.

15. Project Structure

The implemented repository is organized as follows:

```text
PocketSmart_AI_source/
├── app/
│ ├── main.py
│ ├── config.py
│ ├── database.py
│ ├── auth.py
│ ├── models.py
│ ├── schemas.py
│ ├── dependencies.py
│ ├── routers/
│ │ ├── auth.py
│ │ ├── planners.py
│ │ ├── api.py
│ │ └── pages.py
│ ├── services/
│ │ ├── catalog.py
│ │ ├── gemini.py
│ │ └── recommendations.py
│ ├── templates/
│ │ ├── base.html
│ │ ├── index.html
│ │ ├── login.html
│ │ ├── register.html
│ │ ├── dashboard.html
│ │ ├── home.html
. . .
```

```
│   ├── party.html
│   ├── jewelry.html
│   └── history.html
│   └── static/
│   ├── css/style.css
│   └── js/app.js
├── tests/
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```